

```

1 //#####
2 // FILE:   FinalProjectStarter_main.c
3 //
4 // TITLE:  Final Project Starter
5 //#####
6
7 // Included Files
8 #include <stdio.h>
9 #include <stdlib.h>
10 #include <stdarg.h>
11 #include <string.h>
12 #include <math.h>
13 #include <limits.h>
14 #include "F28x_Project.h"
15 #include "F2837xD_SWPrioritizedIsrLevels.h"
16 #include "driverlib.h"
17 #include "device.h"
18 #include "F28379dSerial.h"
19 #include "song.h"
20 #include "dsp.h"
21 #include "fpu32/fpu_rfft.h"
22 #include "xy.h"
23 #include "MatrixMath.h"
24 #include "SE423Lib.h"
25 #include "OptiTrack.h"
26
27 #define PI          3.1415926535897932384626433832795
28 #define TWOPI       6.283185307179586476925286766559
29 #define HALFPi      1.5707963267948966192313216916398
30 // The Launchpad's CPU Frequency set to 200 you should not change this value
31 #define LAUNCHPAD_CPU_FREQUENCY 200
32
33 // ----- code for CAN start here -----
34 #include "F28379dCAN.h"
35 // #define TX_MSG_DATA_LENGTH 4
36 // #define TX_MSG_OBJ_ID 0 //transmit
37
38 #define RX_MSG_DATA_LENGTH 8
39 #define RX_MSG_OBJ_ID_1 1 //measurement from sensor 1
40 #define RX_MSG_OBJ_ID_2 2 //measurement from sensor 2
41 #define RX_MSG_OBJ_ID_3 3 //quality from sensor 1
42 #define RX_MSG_OBJ_ID_4 4 //quality from sensor 2
43 // ----- code for CAN end here -----
44
45 // Interrupt Service Routines predefinition
46 __interrupt void cpu_timer0_isr(void);
47 __interrupt void cpu_timer1_isr(void);
48 __interrupt void cpu_timer2_isr(void);
49 __interrupt void SWI1_HighestPriority(void);
50 __interrupt void SWI2_MiddlePriority(void);
51 __interrupt void SWI3_LowestPriority(void);
52 __interrupt void ADCC_ISR(void);
53 __interrupt void SPIB_isr(void);
54 // ----- code for CAN start here -----
55 __interrupt void can_isr(void);
56 // ----- code for CAN end here -----
57
58 void setF28027EPWM1A(float controleffort);
59 int16_t EPwm1A_F28027 = 1500;
60 void setF28027EPWM2A(float controleffort);
61 int16_t EPwm2A_F28027 = 1500;
62
63 // ----- code for CAN start here -----
64 // volatile uint32_t txMsgCount = 0;
65 // extern uint16_t txMsgData[4];
66
67 volatile uint32_t rxMsgCount_1 = 0;
68 volatile uint32_t rxMsgCount_2 = 0;
69 extern uint16_t rxMsgData[8];
70
71 uint32_t dis_raw_1[2];
72 uint32_t dis_raw_2[2];
73 uint32_t dis_1 = 0;
74 uint32_t dis_2 = 0;
75
76 uint32_t quality_raw_1[4];
77 uint32_t quality_raw_2[4];
78 float quality_1 = 0.0;
79 float quality_2 = 0.0;
80
81 uint32_t lightlevel_raw_1[4];
82 uint32_t lightlevel_raw_2[4];
83 float lightlevel_1 = 0.0;
84 float lightlevel_2 = 0.0;
85
86 uint32_t measure_status_1 = 0;
87 uint32_t measure_status_2 = 0;
88
89 volatile uint32_t errorFlag = 0;
90 // ----- code for CAN end here -----
91
92 uint32_t numTimer0calls = 0;
93 uint16_t UARTPrint = 0;
94
95 float printLV1 = 0;
96 float printLV2 = 0;
97
98 float printLinux1 = 0;
99 float printLinux2 = 0;
100
101 uint16_t LADARi = 0;
102 char G_command[] = "G04472503\n"; //command for getting distance -120 to 120 degree
103 uint16_t G_len = 11; //length of command
104 xy_ladar_pts[228]; //xy data
105 float LADARrightfront = 0;
106 float LADARfront = 0;
107 float LADARtemp_x = 0;
108 float LADARtemp_y = 0;
109 extern datapts_ladar_data[228];
110
111 extern uint16_t newLinuxCommands;
112 extern float LinuxCommands[CMDNUM_FROM_FLOATS];

```

```

113
114 extern uint16_t NewLVData;
115 extern float fromLVvalues[LVNUM_TOFROM_FLOATS];
116 extern LVSendFloats_t DataToLabView;
117 extern char LVsenddata[LVNUM_TOFROM_FLOATS*4+2];
118 extern uint16_t LADARpingpong;
119 extern uint16_t NewCAMDataThreshold1; // Flag new data
120 extern float fromCAMvaluesThreshold1[CAMNUM_FROM_FLOATS];
121 extern uint16_t NewCAMDataThreshold2; // Flag new data
122 extern float fromCAMvaluesThreshold2[CAMNUM_FROM_FLOATS];
123
124 extern uint16_t NewCAMDataAprilTag1; // Flag new data
125 extern float fromCAMvaluesAprilTag1[CAMNUM_FROM_FLOATS];
126
127 float tagid = 0;
128 float tagx = 0;
129 float tagy = 0;
130 float tagz = 0;
131 float tagthetax = 0;
132 float tagthetay = 0;
133 float tagthetaz = 0;
134 int32_t numtag = 0;
135 int32_t AprilTagState = 10;
136 float KpAprilTag = -1.0;
137 float AprilTagSetPoint = 0;
138 int32_t AprilTagCount = 0;
139
140 float MaxAreaThreshold1 = 0;
141 float MaxColThreshold1 = 0;
142 float MaxRowThreshold1 = 0;
143 float NextLargestAreaThreshold1 = 0;
144 float NextLargestColThreshold1 = 0;
145 float NextLargestRowThreshold1 = 0;
146 float NextNextLargestAreaThreshold1 = 0;
147 float NextNextLargestColThreshold1 = 0;
148 float NextNextLargestRowThreshold1 = 0;
149
150 float MaxAreaThreshold2 = 0;
151 float MaxColThreshold2 = 0;
152 float MaxRowThreshold2 = 0;
153 float NextLargestAreaThreshold2 = 0;
154 float NextLargestColThreshold2 = 0;
155 float NextLargestRowThreshold2 = 0;
156 float NextNextLargestAreaThreshold2 = 0;
157 float NextNextLargestColThreshold2 = 0;
158 float NextNextLargestRowThreshold2 = 0;
159
160 float odist = 0.0;
161 float gdist = 0.0;
162
163 float kpvision = 0.0;
164 uint32_t count22 = 0;
165 uint32_t count1 = 0;
166
167 uint32_t numThres1 = 0;
168 uint32_t numThres2 = 0;
169
170 pose ROBOTps = {0,0,0}; //robot position
171 pose LADARps = {3.5/12,0,0,1}; // 3.5/12 for front mounting, theta is not used in this current code
172 float LADARxoffset = 0;
173 float LADARyoffset = 0;
174
175 uint32_t timecount = 0;
176 int16_t RobotState = 1;
177 int16_t checkfronttally = 0;
178 int32_t WallFollowtime = 0;
179
180
181 float colcentroid = 0.0;
182
183 #define NUMWAYPOINTS 10
184 uint16_t statePos = 0;
185 pose robotdest[NUMWAYPOINTS]; // array of waypoints for the robot
186 uint16_t i = 0; //for loop
187
188 uint16_t right_wall_follow_state = 2; // right follow
189 float Kp_front_wall = -2.0;
190 float front_turn_velocity = 0.2;
191 float left_turn_Stop_threshold = 3.5;
192 float Kp_right_wal = -4.0;
193 float ref_right_wall = 1.1;
194 float foward_velocity = 1.0;
195 float left_turn_Start_threshold = 1.3;
196 float turn_saturation = 2.5;
197
198 float x_pred[3][1] = {{0},{0},{0}}; // predicted state
199
200 //more kalman vars
201 float B[3][2] = {{1,0},{1,0},{0,1}}; // control input model
202 float u[2][1] = {{0},{0}}; // control input in terms of velocity and angular velocity
203 float Bu[3][1] = {{0},{0},{0}}; // matrix multiplication of B and u
204 float z[3][1]; // state measurement
205 float eye3[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // 3x3 identity matrix
206 float K[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // optimal Kalman gain
207 #define ProcUncert 0.0001
208 #define CovScalar 10
209 float Q[3][3] = {{ProcUncert,0,ProcUncert/CovScalar},
210                 {0,ProcUncert,ProcUncert/CovScalar},
211                 {ProcUncert/CovScalar,ProcUncert/CovScalar,ProcUncert}}; // process noise (covariance of encoders and gyro)
212 #define MeasUncert 1
213 float R[3][3] = {{MeasUncert,0,MeasUncert/CovScalar},
214                 {0,MeasUncert,MeasUncert/CovScalar},
215                 {MeasUncert/CovScalar,MeasUncert/CovScalar,MeasUncert}}; // measurement noise (covariance of OptiTrack Motion Capture measurement)
216 float S[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // innovation covariance
217 float S_inv[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // innovation covariance matrix inverse
218 float P_pred[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // predicted covariance (measure of uncertainty for current position)
219 float temp_3x3[3][3]; // intermediate storage
220 float temp_3x1[3][1]; // intermediate storage
221 float ytilde[3][1]; // difference between predictions
222
223 // Optitrack Variables
224 int32_t OptiNumUpdatesProcessed = 0;
225 pose OPTITRACKps;
226 extern uint16_t new_optitrack;

```

```

227 extern float Optitrackdata[OPTITRACKDATASIZE];
228 int16_t newOPTITRACKpose=0;
229
230 int16_t adcc2result = 0;
231 int16_t adcc3result = 0;
232 int16_t adcc4result = 0;
233 int16_t adcc5result = 0;
234 float adcC2Volt = 0.0;
235 float adcC3Volt = 0.0;
236 float adcC4Volt = 0.0;
237 float adcC5Volt = 0.0;
238 int32_t numADCCcalls = 0;
239 float LeftWheel = 0;
240 float RightWheel = 0;
241 float LeftWheel_1 = 0;
242 float RightWheel_1 = 0;
243 float LeftVel = 0;
244 float RightVel = 0;
245 float uLeft = 0;
246 float uRight = 0;
247 float HandValue = 0;
248 float vref = 0;
249 float turn = 0;
250 float gyro9250_drift = 0;
251 float gyro9250_angle = 0;
252 float old_gyro9250 = 0;
253 float gyroLPR510_angle = 0;
254 float gyroLPR510_offset = 0;
255 float gyroLPR510_drift = 0;
256 float old_gyroLPR510 = 0;
257 float gyro9250_radians = 0;
258 float gyroLPR510_radians = 0;
259
260 int16_t readdata[25];
261 int16_t IMU_data[9];
262
263 float accelx = 0;
264 float accely = 0;
265 float accelz = 0;
266 float gyrox = 0;
267 float gyroy = 0;
268 float gyroz = 0;
269
270 // Needed global Variables
271 float accelx_offset = 0;
272 float accely_offset = 0;
273 float accelz_offset = 0;
274 float gyrox_offset = 0;
275 float gyroy_offset = 0;
276 float gyroz_offset = 0;
277 int16_t calibration_state = 0;
278 int32_t calibration_count = 0;
279 int16_t doneCal = 0;
280
281 #define MPU9250 1
282 #define DAN28027 2
283 int16_t CurrentChip = MPU9250;
284 int16_t DAN28027Garbage = 0;
285 int16_t dan28027adc1 = 0;
286 int16_t dan28027adc2 = 0;
287 uint16_t MPU9250ignoreCNT = 0; //This is ignoring the first few interrupts if ADCC_ISR and start sending to IMU after these first few interrupts.
288
289 float RCangle;
290
291 void main(void)
292 {
293     // PLL, WatchDog, enable Peripheral Clocks
294     // This example function is found in the F2837xD_SysCtrl.c file.
295     InitSysCtrl();
296
297     InitGpio();
298
299     InitSE423DefaultGPIO();
300
301     //Scope for Timing
302     GPIO_SetupPinMux(11, GPIO_MUX_CPU1, 0);
303     GPIO_SetupPinOptions(11, GPIO_OUTPUT, GPIO_PUSHPULL);
304     GpioDataRegs.GPACLEAR.bit.GPIO11 = 1;
305
306     GPIO_SetupPinMux(32, GPIO_MUX_CPU1, 0);
307     GPIO_SetupPinOptions(32, GPIO_OUTPUT, GPIO_PUSHPULL);
308     GpioDataRegs.GPBCLEAR.bit.GPIO32 = 1;
309
310     GPIO_SetupPinMux(52, GPIO_MUX_CPU1, 0);
311     GPIO_SetupPinOptions(52, GPIO_OUTPUT, GPIO_PUSHPULL);
312     GpioDataRegs.GPBCLEAR.bit.GPIO52 = 1;
313
314     GPIO_SetupPinMux(60, GPIO_MUX_CPU1, 0);
315     GPIO_SetupPinOptions(60, GPIO_OUTPUT, GPIO_PUSHPULL);
316     GpioDataRegs.GPBCLEAR.bit.GPIO60 = 1;
317
318     GPIO_SetupPinMux(61, GPIO_MUX_CPU1, 0);
319     GPIO_SetupPinOptions(61, GPIO_OUTPUT, GPIO_PUSHPULL);
320     GpioDataRegs.GPBCLEAR.bit.GPIO61 = 1;
321
322     GPIO_SetupPinMux(67, GPIO_MUX_CPU1, 0);
323     GPIO_SetupPinOptions(67, GPIO_OUTPUT, GPIO_PUSHPULL);
324     GpioDataRegs.GPCCLEAR.bit.GPIO67 = 1;
325
326 // ----- code for CAN start here -----
327 //GPIO17 - CANRXB
328 GPIO_SetupPinMux(17, GPIO_MUX_CPU1, 2);
329 GPIO_SetupPinOptions(17, GPIO_INPUT, GPIO_ASYNC);
330
331 //GPIO12 - CANTXB
332 GPIO_SetupPinMux(12, GPIO_MUX_CPU1, 2);
333 GPIO_SetupPinOptions(12, GPIO_OUTPUT, GPIO_PUSHPULL);
334 // ----- code for CAN end here -----
335
336
337
338 // ----- code for CAN start here -----
339 // Initialize the CAN controller
340 InitCANB();

```

```

341 // Set up the CAN bus bit rate to 1000 kbps
342 setCANBitRate(200000000, 1000000);
343
344
345 // Enables Interrupt line 0, Error & Status Change interrupts in CAN_CTL register.
346 CanbRegs.CAN_CTL.bit.IE0= 1;
347 CanbRegs.CAN_CTL.bit.EIE= 1;
348 // ----- code for CAN end here -----
349
350 // Clear all interrupts and initialize PIE vector table:
351 // Disable CPU interrupts
352 DINT;
353
354 // Initialize the PIE control registers to their default state.
355 // The default state is all PIE interrupts disabled and flags
356 // are cleared.
357 // This function is found in the F2837xD_PieCtrl.c file.
358 InitPieCtrl();
359
360 // Disable CPU interrupts and clear all CPU interrupt flags:
361 IER = 0x0000;
362 IFR = 0x0000;
363
364 // Initialize the PIE vector table with pointers to the shell Interrupt
365 // Service Routines (ISR).
366 // This will populate the entire table, even if the interrupt
367 // is not used in this example. This is useful for debug purposes.
368 // The shell ISR routines are found in F2837xD_DefaultIsr.c.
369 // This function is found in F2837xD_PieVect.c.
370 InitPieVectTable();
371
372 // Interrupts that are used in this example are re-mapped to
373 // ISR functions found within this project
374 EALLOW; // This is needed to write to EALLOW protected registers
375 PieVectTable.TIMER0_INT = &cpu_timer0_isr;
376 PieVectTable.TIMER1_INT = &cpu_timer1_isr;
377 PieVectTable.TIMER2_INT = &cpu_timer2_isr;
378 PieVectTable.SCIA_RX_INT = &RXAINT_recv_ready;
379 PieVectTable.SCIB_RX_INT = &RXBINT_recv_ready;
380 PieVectTable.SCIC_RX_INT = &RXCINT_recv_ready;
381 PieVectTable.SCID_RX_INT = &RXDINT_recv_ready;
382 PieVectTable.SCIA_TX_INT = &TXAINT_data_sent;
383 PieVectTable.SCIB_TX_INT = &TXBINT_data_sent;
384 PieVectTable.SCIC_TX_INT = &TXCINT_data_sent;
385 PieVectTable.SCID_TX_INT = &TXDINT_data_sent;
386 PieVectTable.ADCC1_INT = &ADCC_ISR;
387 PieVectTable.SPIB_RX_INT = &SPIB_isr;
388 PieVectTable.EMIF_ERROR_INT = &SWI1_HighestPriority; // Using Interrupt12 interrupts that are not used as SWIs
389 PieVectTable.RAM_CORRECTABLE_ERROR_INT = &SWI2_MiddlePriority;
390 PieVectTable.FLASH_CORRECTABLE_ERROR_INT = &SWI3_LowestPriority;
391 // ----- code for CAN start here -----
392 PieVectTable.CANB0_INT = &can_isr;
393 // ----- code for CAN end here -----
394 EDIS; // This is needed to disable write to EALLOW protected registers
395
396 // Initialize the CpuTimers Device Peripheral. This function can be
397 // found in F2837xD_CpuTimers.c
398 InitCpuTimers();
399
400 // Configure CPU-Timer 0, 1, and 2 to interrupt every second:
401 // 200MHz CPU Freq, Period (in uSeconds)
402 ConfigCpuTimer(&CpuTimer0, LAUNCHPAD_CPU_FREQUENCY, 10000); // Currently not used for any purpose
403 ConfigCpuTimer(&CpuTimer1, LAUNCHPAD_CPU_FREQUENCY, 100000); // !!!!! Important, Used to command LADAR every 100ms. Do not Change.
404 ConfigCpuTimer(&CpuTimer2, LAUNCHPAD_CPU_FREQUENCY, 40000); // Currently not used for any purpose
405
406 // Enable CpuTimer Interrupt bit TIE
407 CpuTimer0Regs.TCR.all = 0x4000;
408 CpuTimer1Regs.TCR.all = 0x4000;
409 CpuTimer2Regs.TCR.all = 0x4000;
410
411 DELAY_US(1000000); // Delay 1 second giving Lidar Time to power on after system power on
412
413 init_serialSCIB(&SerialB,19200);
414 init_serialSCIC(&SerialC,19200);
415
416 for (LADARi = 0; LADARi < 228; LADARi++) {
417     ladar_data[LADARi].angle = ((3*LADARi+44)*0.3515625-135)*0.01745329; //0.017453292519943 is pi/180, multiplication is faster; 0.3515625 is 360/1024
418 }
419
420 init_eQEPs();
421 init_EPWM1and2();
422 init_RCServoPWM_3AB_5AB_6A();
423 init_ADCsAndDACs();
424 setupSpib();
425
426 EALLOW;
427 EPwm4Regs.ETSEL.bit.SOCAEN = 0; // Disable SOCA on A group
428 EPwm4Regs.TBCTL.bit.CTRMODE = 3; // freeze counter
429 EPwm4Regs.ETSEL.bit.SOCASEL = 2; // Select Event when counter equal to PRD
430 EPwm4Regs.ETPS.bit.SOCAPRD = 1; // Generate pulse on 1st event
431 EPwm4Regs.TBCTR = 0x0; // Clear counter
432 EPwm4Regs.TBPHS.bit.TBPHS = 0x0000; // Phase is 0
433 EPwm4Regs.TBCTL.bit.PHSEN = 0; // Disable phase loading
434 EPwm4Regs.TBCTL.bit.CLKDIV = 0; // divide by 1 50Mhz Clock
435 EPwm4Regs.TBPRD = 50000; // Set Period to 1ms sample. Input clock is 50MHz.
436 // Notice here that we are not setting CMPA or CMPB because we are not using the PWM signal
437 EPwm4Regs.ETSEL.bit.SOCAEN = 1; //enable SOCA
438 //EPwm4Regs.TBCTL.bit.CTRMODE = 0; //unfreeze, wait to do this right before enabling interrupts
439 EDIS;
440
441 // Enable CPU int1 which is connected to CPU-Timer 0, CPU int13
442 // which is connected to CPU-Timer 1, and CPU int 14, which is connected
443 // to CPU-Timer 2: int 12 is for the SWI.
444 IER |= M_INT1;
445 IER |= M_INT6;
446 IER |= M_INT8; // SCIC SCID
447 IER |= M_INT9; // SCIA
448 IER |= M_INT12;
449 IER |= M_INT13;
450 IER |= M_INT14;
451
452 // Enable TINT0 in the PIE: Group 1 interrupt 7
453 PieCtrlRegs.PIEIER1.bit.INTx7 = 1;
454 PieCtrlRegs.PIEIER1.bit.INTx3 = 1;

```

```

455 PieCtrlRegs.PIEIER6.bit.INTx3 = 1;
456 PieCtrlRegs.PIEIER12.bit.INTx9 = 1; //SWI1
457 PieCtrlRegs.PIEIER12.bit.INTx10 = 1; //SWI2
458 PieCtrlRegs.PIEIER12.bit.INTx11 = 1; //SWI3 Lowest priority
459 // ----- code for CAN start here -----
460 // Enable CANB in the PIE: Group 9 interrupt 7
461 PieCtrlRegs.PIEIER9.bit.INTx7 = 1;
462 // ----- code for CAN end here -----
463
464 // ----- code for CAN start here -----
465 // Enable the CAN interrupt signal
466 CanbRegs.CAN_GLB_INT_EN.bit.GLBINT0_EN = 1;
467 // ----- code for CAN end here -----
468
469
470 robotdest[0].x = -4;    robotdest[0].y = 10;
471 robotdest[1].x = -4;    robotdest[1].y = 2;
472 //middle of bottom
473 robotdest[2].x = 0;     robotdest[2].y = 2;
474 //outside the course
475 robotdest[3].x = 0;     robotdest[3].y = -3;
476 //back to middle
477 robotdest[4].x = 0;     robotdest[4].y = 2;
478 robotdest[5].x = 4;     robotdest[5].y = 2;
479 robotdest[6].x = 4;     robotdest[6].y = 10;
480 robotdest[7].x = 0;     robotdest[7].y = 9;
481 robotdest[8].x = 4;     robotdest[8].y = 5;
482 robotdest[9].x = 2;     robotdest[9].y = 8;
483
484 // ROBOTps will be updated by Optitrack during gyro calibration
485 // TODO: specify the starting position of the robot
486 ROBOTps.x = 0;          //the estimate in array form (useful for matrix operations)
487 ROBOTps.y = 0;
488 ROBOTps.theta = 0; // was -PI: need to flip OT ground plane to fix this
489 x_pred[0][0] = ROBOTps.x; //estimate in structure form (useful elsewhere)
490 x_pred[1][0] = ROBOTps.y;
491 x_pred[2][0] = ROBOTps.theta;
492
493 AdccRegs.ADCINTFLGCLR.bit.ADCINT1 = 1; //clear interrupt flag
494 PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
495 SpibRegs.SPIFFRX.bit.RXFFOVFCLR=1; // Clear Overflow flag
496 SpibRegs.SPIFFRX.bit.RXFFINTCLR=1; // Clear Interrupt flag
497 PieCtrlRegs.PIEACK.all = PIEACK_GROUP6;
498 EPwm4Regs.TBCTL.bit.CTRMODE = 0; //unfreeze, and enter up count mode
499
500 init_serialSCIA(&SerialA,115200);
501 init_serialSCID(&SerialD,2083332);
502
503 // Enable global Interrupts and higher priority real-time debug events
504 EINT; // Enable Global interrupt INTM
505 ERTM; // Enable Global realtime interrupt DBGM
506 // ----- code for CAN start here -----
507 // Measured Distance from 1
508 // Initialize the receive message object 1 used for receiving CAN messages.
509 // Message Object Parameters:
510 //   Message Object ID Number: 1
511 //   Message Identifier: 0x060b0101
512 //   Message Frame: Standard
513 //   Message Type: Receive
514 //   Message ID Mask: 0x0
515 //   Message Object Flags: Receive Interrupt
516 //   Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
517 //   for a Receive mailbox)
518 //
519 CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_1, 0x060b0101, CAN_MSG_FRAME_EXT,
520                       CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
521                       RX_MSG_DATA_LENGTH);
522
523 // Measured Distance from 2
524 // Initialize the receive message object 2 used for receiving CAN messages.
525 // Message Object Parameters:
526 //   Message Object ID Number: 2
527 //   Message Identifier: 0x060b0102
528 //   Message Frame: Standard
529 //   Message Type: Receive
530 //   Message ID Mask: 0x0
531 //   Message Object Flags: Receive Interrupt
532 //   Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
533 //   for a Receive mailbox)
534 //
535
536 CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_2, 0x060b0102, CAN_MSG_FRAME_EXT,
537                       CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
538                       RX_MSG_DATA_LENGTH);
539
540 // Measurement Quality from 1
541 // Initialize the receive message object 2 used for receiving CAN messages.
542 // Message Object Parameters:
543 //   Message Object ID Number: 3
544 //   Message Identifier: 0x060b0201
545 //   Message Frame: Standard
546 //   Message Type: Receive
547 //   Message ID Mask: 0x0
548 //   Message Object Flags: Receive Interrupt
549 //   Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
550 //   for a Receive mailbox)
551 //
552
553 CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_3, 0x060b0201, CAN_MSG_FRAME_EXT,
554                       CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
555                       RX_MSG_DATA_LENGTH);
556
557 // Measurement Quality from 2
558 // Initialize the receive message object 2 used for receiving CAN messages.
559 // Message Object Parameters:
560 //   Message Object ID Number: 4
561 //   Message Identifier: 0x060b0202
562 //   Message Frame: Standard
563 //   Message Type: Receive
564 //   Message ID Mask: 0x0
565 //   Message Object Flags: Receive Interrupt
566 //   Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
567 //   for a Receive mailbox)
568 //

```

```

569 CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_4, 0x060b0202, CAN_MSG_FRAME_EXT,
570                       CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
571                       RX_MSG_DATA_LENGTH);
572
573
574 //
575 // Start CAN module operations
576 //
577 CanbRegs.CAN_CTL.bit.Init = 0;
578 CanbRegs.CAN_CTL.bit.CCE = 0;
579
580
581 // ----- code for CAN end here -----
582 char S_command[19] = "S1152000124000\n"; // this change the baud rate to 115200
583 uint16_t S_len = 19;
584 serial_sendSCIC(&SerialC, S_command, S_len);
585
586
587 DELAY_US(1000000); // Delay letting Lidar change its Baud rate
588 init_serialSCIC(&SerialC, 115200);
589
590
591 // LED1 is GPIO22
592 GpioDataRegs.GPACLEAR.bit.GPIO22 = 1;
593 // LED2 is GPIO94
594 GpioDataRegs.GPCCLEAR.bit.GPIO94 = 1;
595 // LED3 is GPIO95
596 GpioDataRegs.GPCCLEAR.bit.GPIO95 = 1;
597 // LED4 is GPIO97
598 GpioDataRegs.GPDCLEAR.bit.GPIO97 = 1;
599 // LED5 is GPIO111
600 GpioDataRegs.GPDCLEAR.bit.GPIO111 = 1;
601
602
603 // IDLE loop. Just sit and loop forever (optional):
604 while(1)
605 {
606     if (UARTPrint == 1) {
607
608         if (readbuttons() == 0) {
609             UART_printfLine(1, "State:%d : %d", RobotState, statePos);
610             UART_printfLine(2, "0 %.3f G %.3f", odist, gdist);
611             //UART_printfLine(1, "Vrf:%.2f trn:%.2f", vref, turn);
612             //UART_printfLine(1, "x:%.2f y:%.2f a:%.2f", ROBOTps.x, ROBOTps.y, ROBOTps.theta);
613             UART_printfLine(2, "F%.4f R%.4f", LADARfront, LADARrightfront);
614         } else if (readbuttons() == 1) {
615             UART_printfLine(1, "01A:%.0fC:%.0fR:%.0f", MaxAreaThreshold1, MaxColThreshold1, MaxRowThreshold1);
616             UART_printfLine(2, "P1A:%.0fC:%.0fR:%.0f", MaxAreaThreshold2, MaxColThreshold2, MaxRowThreshold2);
617             //UART_printfLine(1, "LV1:%.3f LV2:%.3f", printLV1, printLV2);
618             //UART_printfLine(2, "Ln1:%.3f Ln2:%.3f", printLinux1, printLinux2);
619         } else if (readbuttons() == 2) {
620             UART_printfLine(1, "02A:%.0fC:%.0fR:%.0f", NextLargestAreaThreshold1, NextLargestColThreshold1, NextLargestRowThreshold1);
621             UART_printfLine(2, "P2A:%.0fC:%.0fR:%.0f", NextLargestAreaThreshold2, NextLargestColThreshold2, NextLargestRowThreshold2);
622             // UART_printfLine(1, "%.2f %.2f", adcC2Volt, adcC3Volt);
623             // UART_printfLine(2, "%.2f %.2f", adcC4Volt, adcC5Volt);
624         } else if (readbuttons() == 4) {
625             UART_printfLine(1, "03A:%.0fC:%.0fR:%.0f", NextNextLargestAreaThreshold1, NextNextLargestColThreshold1, NextNextLargestRowThreshold1);
626             UART_printfLine(2, "P3A:%.0fC:%.0fR:%.0f", NextNextLargestAreaThreshold2, NextNextLargestColThreshold2, NextNextLargestRowThreshold2);
627             // UART_printfLine(1, "L:%.3f R:%.3f", LeftVel, RightVel);
628             // UART_printfLine(2, "uL:%.2f uR:%.2f", uLeft, uRight);
629         } else if (readbuttons() == 8) {
630             UART_printfLine(1, "020x%.2f y%.2f", ladar_pts[20].x, ladar_pts[20].y);
631             UART_printfLine(2, "150x%.2f y%.2f", ladar_pts[150].x, ladar_pts[150].y);
632         } else if (readbuttons() == 3) {
633             UART_printfLine(1, "Vrf:%.2f trn:%.2f", vref, turn);
634             UART_printfLine(2, "MPU:%.2f LPR:%.2f", gyro9250_radians, gyroLPR510_radians);
635         } else if (readbuttons() == 5) {
636             UART_printfLine(1, "0x:%.2f 0y:%.2f 0a:%.2f", OPTITRACKps.x, OPTITRACKps.y, OPTITRACKps.theta);
637             UART_printfLine(2, "State:%d : %d", RobotState, statePos);
638         } else if (readbuttons() == 6) {
639             UART_printfLine(1, "D1 %ld D2 %ld", dis_1, dis_2);
640             UART_printfLine(2, "St1 %ld St2 %ld", measure_status_1, measure_status_2);
641         } else if (readbuttons() == 7) {
642             UART_printfLine(1, "%.0f, %.1f, %.1f, %.1f", tagid, tagx, tagy, tagz);
643             UART_printfLine(2, "%.1f, %.1f, %.1f", tagthetax, tagthetay, tagthetaz);
644         }
645         //UART_printfLine(1, "RCangle %.1f ", RCangle);
646
647         UARTPrint = 0;
648     }
649 }
650 }
651
652 __interrupt void SPIB_isr(void){
653
654     uint16_t i;
655     GpioDataRegs.GPBSET.bit.GPIO32 = 1;
656
657     GpioDataRegs.GPCSET.bit.GPIO66 = 1; //Pull CS high as done R/W
658     GpioDataRegs.GPASET.bit.GPIO29 = 1; // Pull CS high for DAN28027
659
660     if (CurrentChip == MPU9250) {
661
662         for (i=0; i<8; i++) {
663             readdata[i] = SpibRegs.SPIRXBUF; // readdata[0] is garbage
664         }
665
666         PostSWI1(); // Manually cause the interrupt for the SWI1
667
668     } else if (CurrentChip == DAN28027) {
669
670         DAN28027Garbage = SpibRegs.SPIRXBUF;
671         dan28027adc1 = SpibRegs.SPIRXBUF;
672         dan28027adc2 = SpibRegs.SPIRXBUF;
673         CurrentChip = MPU9250;
674         SpibRegs.SPIFFCT.bit.TXDLY = 0;
675     }
676
677     SpibRegs.SPIFFRX.bit.RXFFOVFLCLR=1; // Clear Overflow flag
678     SpibRegs.SPIFFRX.bit.RXFFINTCLR=1; // Clear Interrupt flag
679     PieCtrlRegs.PIEACK.all = PIEACK_GROUP6;
680     GpioDataRegs.GPBCLEAR.bit.GPIO32 = 1;
681 }
682

```



```

683
684 //adcd1 pie interrupt
685 __interrupt void ADCC_ISR (void)
686 {
687     GpioDataRegs.GPASET.bit.GPIO11 = 1;
688     adcc2result = AdccResultRegs.ADCRESULT0;
689     adcc3result = AdccResultRegs.ADCRESULT1;
690     adcc4result = AdccResultRegs.ADCRESULT2;
691     adcc5result = AdccResultRegs.ADCRESULT3;
692
693     // Here covert ADCIND0 to volts
694     adcC2Volt = adcc2result*3.0/4095.0;
695     adcC3Volt = adcc3result*3.0/4095.0;
696     adcC4Volt = adcc4result*3.0/4095.0;
697     adcC5Volt = adcc5result*3.0/4095.0;
698
699     if (MPU9250ignoreCNT >= 1) {
700         CurrentChip = MPU9250;
701         SpibRegs.SPIFFCT.bit.TXDLY = 0;
702         SpibRegs.SPIFFRX.bit.RXFFIL = 8;
703
704         GpioDataRegs.GPCCLEAR.bit.GPIO66 = 1;
705
706         SpibRegs.SPITXBUF = ((0x8000)|(0x3A00));
707         SpibRegs.SPITXBUF = 0;
708         SpibRegs.SPITXBUF = 0;
709         SpibRegs.SPITXBUF = 0;
710         SpibRegs.SPITXBUF = 0;
711         SpibRegs.SPITXBUF = 0;
712         SpibRegs.SPITXBUF = 0;
713         SpibRegs.SPITXBUF = 0;
714     } else {
715         MPU9250ignoreCNT++;
716     }
717
718     numADCCcalls++;
719     AdccRegs.ADCINTFLGCLR.bit.ADCINT1 = 1; //clear interrupt flag
720     PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
721     GpioDataRegs.GPACLEAR.bit.GPIO11 = 1;
722 }
723 }
724
725 // cpu_timer0_isr - CPU Timer0 ISR
726 __interrupt void cpu_timer0_isr(void)
727 {
728     CpuTimer0.InterruptCount++;
729
730     numTimer0calls++;
731
732     if ((numTimer0calls%5) == 0) {
733         // Blink LaunchPad Red LED
734         //GpioDataRegs.GPBTOGGLE.bit.GPIO34 = 1;
735     }
736
737     // Acknowledge this interrupt to receive more interrupts from group 1
738     PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
739 }
740
741 // cpu_timer1_isr - CPU Timer1 ISR
742 __interrupt void cpu_timer1_isr(void)
743 {
744
745     serial_sendSCIC(&SerialC, G_command, G_len);
746
747     CpuTimer1.InterruptCount++;
748 }
749
750 // cpu_timer2_isr CPU Timer2 ISR
751 __interrupt void cpu_timer2_isr(void)
752 {
753     // Blink LaunchPad Blue LED
754     //GpioDataRegs.GPATOGGLE.bit.GPIO31 = 1;
755
756     CpuTimer2.InterruptCount++;
757
758     PostSWI3();
759
760
761     // if ((CpuTimer2.InterruptCount % 10) == 0) {
762     //     UARTPrint = 1;
763     // }
764 }
765
766 void setF28027EPWM1A(float controleffort){
767     if (controleffort < -10) {
768         controleffort = -10;
769     }
770     if (controleffort > 10) {
771         controleffort = 10;
772     }
773     float value = (controleffort+10)*3000.0/20.0;
774     EPwm1A_F28027 = (int16_t)value; // Set global variable that is sent over SPI to F28027
775 }
776 void setF28027EPWM2A(float controleffort){
777     if (controleffort < -10) {
778         controleffort = -10;
779     }
780     if (controleffort > 10) {
781         controleffort = 10;
782     }
783     float value = (controleffort+10)*3000.0/20.0;
784     EPwm2A_F28027 = (int16_t)value; // Set global variable that is sent over SPI to F28027
785 }
786
787 //
788 // Connected to PIEIER12_9 (use MINT12 and MG12_9 masks):
789 //
790 __interrupt void SWI1_HighestPriority(void) // EMIF_ERROR
791 {
792     GpioDataRegs.GPASET.bit.GPIO61 = 1;
793     // Set interrupt priority:
794     volatile Uint16 TempPIEIER = PieCtrlRegs.PIEIER12.all;
795     IER |= M_INT12;
796     IER &= MINT12; // Set "global" priority

```

```

797 PieCtrlRegs.PIEIER12.all &= MG12_9; // Set "group" priority
798 PieCtrlRegs.PIEACK.all = 0xFFFF; // Enable PIE interrupts
799 __asm(" NOP");
800 EINT;
801
802 //#####
803 IMU_data[0] = readdata[1];
804 IMU_data[1] = readdata[2];
805 IMU_data[2] = readdata[3];
806 IMU_data[3] = readdata[5];
807 IMU_data[4] = readdata[6];
808 IMU_data[5] = readdata[7];
809
810 accelx = (((float)(IMU_data[0]))*4.0/32767.0);
811 accely = (((float)(IMU_data[1]))*4.0/32767.0);
812 accelz = (((float)(IMU_data[2]))*4.0/32767.0);
813 gyro_x = (((float)(IMU_data[3]))*250.0/32767.0);
814 gyro_y = (((float)(IMU_data[4]))*250.0/32767.0);
815 gyro_z = (((float)(IMU_data[5]))*250.0/32767.0);
816
817 if(calibration_state == 0){
818     calibration_count++;
819     if (calibration_count == 2000) {
820         calibration_state = 1;
821         calibration_count = 0;
822     }
823 } else if(calibration_state == 1){
824     accelx_offset+=accelx;
825     accely_offset+=accely;
826     accelz_offset+=accelz;
827     gyro_x_offset+=gyro_x;
828     gyro_y_offset+=gyro_y;
829     gyro_z_offset+=gyro_z;
830     gyroLPR510_offset+=adcC5Volt;
831     calibration_count++;
832     if (calibration_count == 2000) {
833         calibration_state = 2;
834         accelx_offset/=2000.0;
835         accely_offset/=2000.0;
836         accelz_offset/=2000.0;
837         gyro_x_offset/=2000.0;
838         gyro_y_offset/=2000.0;
839         gyro_z_offset/=2000.0;
840         gyroLPR510_offset/=2000.0;
841         calibration_count = 0;
842     }
843 } else if(calibration_state == 2) {
844
845     gyroLPR510_drift += (((adcC5Volt-gyroLPR510_offset) + old_gyroLPR510)*.0005)/(2000);
846     old_gyroLPR510 = adcC5Volt-gyroLPR510_offset;
847
848     gyro9250_drift += (((gyroz-gyroz_offset) + old_gyro9250)*.0005)/(2000);
849     old_gyro9250 = gyroz-gyroz_offset;
850     calibration_count++;
851
852     if (calibration_count == 2000) {
853         calibration_state = 3;
854         calibration_count = 0;
855         doneCal = 1;
856         newOPTITRACKpose = 0;
857     }
858 } else if(calibration_state == 3){
859     accelx -=(accelx_offset);
860     accely -=(accely_offset);
861     accelz -=(accelz_offset);
862     gyro_x -= gyro_x_offset;
863     gyro_y -= gyro_y_offset;
864     gyro_z -= gyro_z_offset;
865     LeftWheel = readEncLeft();
866     RightWheel = readEncRight();
867     HandValue = readEncWheel();
868
869     gyro9250_angle = gyro9250_angle + (gyroz + old_gyro9250)*.0005 - gyro9250_drift;
870     old_gyro9250 = gyroz;
871
872     gyroLPR510_angle = gyroLPR510_angle + ((adcC5Volt-gyroLPR510_offset) + old_gyroLPR510)*.0005 - gyroLPR510_drift;
873     old_gyroLPR510 = adcC5Volt-gyroLPR510_offset;
874
875     gyro9250_radians = (gyro9250_angle * (PI/180.0));
876     gyroLPR510_radians = gyroLPR510_angle * 400 * (PI/180.0);
877
878     LeftVel = (1.235/12.0)*(LeftWheel - LeftWheel_1)*1000;
879     RightVel = (1.235/12.0)*(RightWheel - RightWheel_1)*1000;
880
881     if (newLinuxCommands == 1) {
882         newLinuxCommands = 0;
883         printLinux1 = LinuxCommands[0];
884         printLinux2 = LinuxCommands[1];
885         /// = LinuxCommands[2];
886         /// = LinuxCommands[3];
887         /// = LinuxCommands[4];
888         /// = LinuxCommands[5];
889         /// = LinuxCommands[6];
890         /// = LinuxCommands[7];
891         /// = LinuxCommands[8];
892         /// = LinuxCommands[9];
893         /// = LinuxCommands[10];
894     }
895
896     if (NewCAMDataThreshold1 == 1) {
897         NewCAMDataThreshold1 = 0;
898         MaxAreaThreshold1 = fromCAMvaluesThreshold1[0];
899         MaxColThreshold1 = fromCAMvaluesThreshold1[1];
900         MaxRowThreshold1 = fromCAMvaluesThreshold1[2];
901
902         NextLargestAreaThreshold1 = fromCAMvaluesThreshold1[3];
903         NextLargestColThreshold1 = fromCAMvaluesThreshold1[4];
904         NextLargestRowThreshold1 = fromCAMvaluesThreshold1[5];
905
906         NextNextLargestAreaThreshold1 = fromCAMvaluesThreshold1[6];
907         NextNextLargestColThreshold1 = fromCAMvaluesThreshold1[7];
908         NextNextLargestRowThreshold1 = fromCAMvaluesThreshold1[8];
909         numThres1++;
910         float mr = MaxRowThreshold1;

```



```

911         odist = -1.15291070692441e-05*pow(mr,3)+0.00348114123702629*pow(mr,2)-0.365334496447587*pow(mr,1)+13.7577551982332+0.75;
912     if ((numThres1 % 5) == 0) {
913         // LED4 is GPIO97
914         GpioDataRegs.GPDT0GGLE.bit.GPIO97 = 1;
915     }
916
917     if ((numThres1 % 100) == 0) {
918         UARTPrint = 1;
919     }
920 }
921
922 if (NewCAMDataThreshold2 == 1) {
923     NewCAMDataThreshold2 = 0;
924     MaxAreaThreshold2 = fromCAMvaluesThreshold2[0];
925     MaxColThreshold2 = fromCAMvaluesThreshold2[1];
926     MaxRowThreshold2 = fromCAMvaluesThreshold2[2];
927
928     NextLargestAreaThreshold2 = fromCAMvaluesThreshold2[3];
929     NextLargestColThreshold2 = fromCAMvaluesThreshold2[4];
930     NextLargestRowThreshold2 = fromCAMvaluesThreshold2[5];
931
932     NextNextLargestAreaThreshold2 = fromCAMvaluesThreshold2[6];
933     NextNextLargestColThreshold2 = fromCAMvaluesThreshold2[7];
934     NextNextLargestRowThreshold2 = fromCAMvaluesThreshold2[8];
935     numThres2++;
936     float mr = MaxRowThreshold2;
937
938     gdist = -1.15291070692441e-05*pow(mr,3)+0.00348114123702629*pow(mr,2)-0.365334496447587*pow(mr,1)+13.7577551982332+0.75;
939
940     if ((numThres2 % 5) == 0) {
941         // LED5 is GPIO111
942         GpioDataRegs.GPDT0GGLE.bit.GPIO111 = 1;
943     }
944     if ((numThres2 % 100) == 0) {
945         UARTPrint = 1;
946     }
947 }
948
949 if (NewCAMDataAprilTag1 == 1) {
950     NewCAMDataAprilTag1 = 0;
951     tagx = fromCAMvaluesAprilTag1[0];
952     tagy = fromCAMvaluesAprilTag1[1];
953     tagz = fromCAMvaluesAprilTag1[2];
954
955     tagthetax = fromCAMvaluesAprilTag1[3];
956     tagthetay = fromCAMvaluesAprilTag1[4];
957     tagthetaz = fromCAMvaluesAprilTag1[5];
958
959     tagid = fromCAMvaluesAprilTag1[6];
960
961     numtag++;
962     if ((numtag % 5) == 0) {
963         // LED2 is GPIO94
964         GpioDataRegs.GPCT0GGLE.bit.GPIO94 = 1;
965     }
966 }
967
968 if (NewLVData == 1) {
969     NewLVData = 0;
970     printLV1 = fromLVvalues[0];
971     printLV2 = fromLVvalues[1];
972     /// = fromLVvalues[2];
973     /// = fromLVvalues[3];
974     /// = fromLVvalues[4];
975     /// = fromLVvalues[5];
976     /// = fromLVvalues[6];
977     /// = fromLVvalues[7];
978 }
979
980
981 if (new_optitrack == 1) {
982     OPTITRACKps = UpdateOptitrackStates(ROBOTps, &newOPTITRACKpose);
983     new_optitrack = 0;
984 }
985
986 // Step 0: update B, u
987 B[0][0] = cosf(ROBOTps.theta)*0.001;
988 B[1][0] = sinf(ROBOTps.theta)*0.001;
989 B[2][1] = 0.001;
990 u[0][0] = 0.5*(LeftVel + RightVel); // linear velocity of robot
991 u[1][0] = gyro*(PI/180.0); // angular velocity in rad/s (negative for right hand angle)
992
993 // Step 1: predict the state and estimate covariance
994 Matrix3x2_Mult(B, u, Bu); // Bu = B*u
995 Matrix3x1_Add(x_pred, Bu, x_pred, 1.0, 1.0); // x_pred = x_pred(old) + Bu
996 Matrix3x3_Add(P_pred, Q, P_pred, 1.0, 1.0); // P_pred = P_pred(old) + Q
997 // Step 2: if there is a new measurement, then update the state
998 if (1 == newOPTITRACKpose) {
999     OptiNumUpdatesProcessed++; // checking how often OptiTrack is causing a Kalman update
1000     if ((OptiNumUpdatesProcessed%10)==0) {
1001         // LED 3
1002         GpioDataRegs.GPCT0GGLE.bit.GPIO95 = 1;
1003     }
1004     newOPTITRACKpose = 0;
1005     z[0][0] = OPTITRACKps.x; // take in the Optitrack Motion Capture measurement
1006     z[1][0] = OPTITRACKps.y;
1007     // fix for OptiTrack problem at 180 degrees
1008     if (cosf(ROBOTps.theta) < -0.99) {
1009         z[2][0] = ROBOTps.theta;
1010     }
1011     else {
1012         z[2][0] = OPTITRACKps.theta;
1013     }
1014     // Step 2a: calculate the innovation/measurement residual, ytilde
1015     Matrix3x1_Add(z, x_pred, ytilde, 1.0, -1.0); // ytilde = z-x_pred
1016     // Step 2b: calculate innovation covariance, S
1017     Matrix3x3_Add(P_pred, R, S, 1.0, 1.0); // S = P_pred + R
1018     // Step 2c: calculate the optimal Kalman gain, K
1019     Matrix3x3_Invert(S, S_inv);
1020     Matrix3x3_Mult(P_pred, S_inv, K); // K = P_pred*(S^-1)
1021     // Step 2d: update the state estimate x_pred = x_pred(old) + K*ytilde
1022     Matrix3x1_Mult(K, ytilde, temp_3x1);
1023     Matrix3x1_Add(x_pred, temp_3x1, x_pred, 1.0, 1.0);
1024     // Step 2e: update the covariance estimate P_pred = (I-K)*P_pred(old)

```

```

1025     Matrix3x3_Add(eye3, K, temp_3x3, 1.0, -1.0);
1026     Matrix3x3_Mult(temp_3x3, P_pred, P_pred);
1027 } // end of correction step
1028
1029 // set ROBOTps to the updated and corrected Kalman values.
1030 ROBOTps.x = x_pred[0][0];
1031 ROBOTps.y = x_pred[1][0];
1032 ROBOTps.theta = x_pred[2][0];
1033
1034 // Make sure this function is called every time in this function even if you decide not to use its vref and turn
1035 // uses xy code to step through an array of positions
1036 if( xy_control(&vref, &turn, 1.0, ROBOTps.x, ROBOTps.y, robotdest[statePos].x, robotdest[statePos].y, ROBOTps.theta, 0.25, 0.5)) {
1037     statePos = (statePos+1)%NUMWAYPOINTS;
1038 }
1039 // state machine
1040 //RobotState = 20;
1041 switch (RobotState) {
1042 case 1:
1043
1044     // vref and turn are the vref and turn returned from xy_control
1045
1046     if (LADARfront < 1.2) {
1047         vref = 0.2;
1048         checkfronttally++;
1049         if (checkfronttally > 310) { // check if LADARfront < 1.2 for 310ms or 3 LADAR samples
1050             RobotState = 10; // Wall follow
1051             WallFollowtime = 0;
1052             right_wall_follow_state = 1;
1053         }
1054     } else {
1055         checkfronttally = 0;
1056     }
1057
1058     count1 += 1;
1059     if (count1 > 2000) {
1060         if (MaxAreaThreshold1 > 35 & (MaxAreaThreshold1 > MaxAreaThreshold2)) {
1061             RobotState=20;
1062         } else if (MaxAreaThreshold2 > 35 & (MaxAreaThreshold2 > MaxAreaThreshold1)) {
1063             RobotState=30;
1064         }
1065     }
1066     break;
1067 case 10:
1068     if (right_wall_follow_state == 1) {
1069         //Left Turn
1070         turn = Kp_front_wall*(14.5 - LADARfront);
1071         vref = front_turn_velocity;
1072         if (LADARfront > left_turn_Stop_threshold) {
1073             right_wall_follow_state = 2;
1074         }
1075     } else if (right_wall_follow_state == 2) {
1076         //Right Wall Follow
1077         turn = Kp_right_wal*(ref_right_wall - LADARrightfront);
1078         vref = foward_velocity;
1079         if (LADARfront < left_turn_Start_threshold) {
1080             right_wall_follow_state = 1;
1081         }
1082     }
1083     if (turn > turn_saturation) {
1084         turn = turn_saturation;
1085     }
1086     if (turn < -turn_saturation) {
1087         turn = -turn_saturation;
1088     }
1089
1090     WallFollowtime++;
1091     if ( (WallFollowtime > 5000) && (LADARfront > 1.5) ) {
1092         RobotState = 1;
1093         checkfronttally = 0;
1094     }
1095     break;
1096 case 20:
1097     // put vision code here
1098     kpvision = -0.05;
1099     colcentroid = MaxColThreshold1 - 80;
1100     if (MaxColThreshold1 == 0 || MaxAreaThreshold1 < 3) {
1101         vref = 0;
1102         turn = 0;
1103     } else {
1104         vref = 0.75;
1105         turn = kpvision*(0-colcentroid);
1106     }
1107
1108     if (MaxRowThreshold1 > 115) {
1109         RobotState = 22;
1110         count22 = 0;
1111     }
1112     break;
1113 case 22:
1114     vref = 0;
1115     turn = 0;
1116     count22 += 1;
1117     if (count22 > 1000) {
1118         RobotState = 24;
1119         count22 = 0;
1120     }
1121     break;
1122 case 24:
1123     vref = 0.5;
1124     turn = 0;
1125     count22 += 1;
1126     if (count22 > 1000) {
1127         RobotState = 26;
1128         count22 = 0;
1129     }
1130     break;
1131 case 26:
1132     vref = 0;
1133     turn = 0;
1134     count22 += 1;
1135     if (count22 > 1000) {
1136         RobotState = 1;
1137         count22 = 0;
1138     }

```

```

1139         count1 = 0;
1140     }
1141     break;
1142
1143
1144     case 30:
1145         // put vision code here
1146         kpvision = -0.05;
1147         colcentroid = MaxColThreshold2 - 80;
1148         if (MaxColThreshold2 == 0 || MaxAreaThreshold2 < 3) {
1149             vref = 0;
1150             turn = 0;
1151         } else {
1152             vref = 0.75;
1153             turn = kpvision*(0-colcentroid);
1154         }
1155
1156         if (MaxRowThreshold2 > 100) {
1157             RobotState = 32;
1158             count22 = 0;
1159         }
1160         break;
1161     case 32:
1162         vref = 0;
1163         turn = 0;
1164         count22 += 1;
1165         if (count22 > 1000) {
1166             RobotState = 34;
1167             count22 = 0;
1168         }
1169         break;
1170     case 34:
1171         vref = 0.5;
1172         turn = 0;
1173         count22 += 1;
1174         if (count22 > 1000) {
1175             RobotState = 36;
1176             count22 = 0;
1177         }
1178         break;
1179     case 36:
1180         vref = 0;
1181         turn = 0;
1182         count22 += 1;
1183         if (count22 > 1000) {
1184             RobotState = 1;
1185             count22 = 0;
1186             count1 = 0;
1187         }
1188         break;
1189     default:
1190         break;
1191 }
1192
1193
1194 //Must be called each time into this SWI1 function
1195 PControl(&uLeft,&uRight,vref,turn,LeftWheel,RightWheel);
1196 // These below lines also must be called each time into this SWI1 function
1197 setEPWM1A(uLeft);
1198 setEPWM2A(-uRight);
1199 setF28027EPWM1A(uLeft);
1200 setF28027EPWM2A(-uRight);
1201 LeftWheel_1 = LeftWheel;
1202 RightWheel_1 = RightWheel;
1203
1204 if((timecount%250) == 0) {
1205     DataToLabView.floatData[0] = ROBOTps.x;
1206     DataToLabView.floatData[1] = ROBOTps.y;
1207     DataToLabView.floatData[2] = ROBOTps.theta;
1208     DataToLabView.floatData[3] = (float)timecount;
1209     DataToLabView.floatData[4] = 0.5*(LeftVel + RightVel);
1210     DataToLabView.floatData[5] = (float)RobotState;
1211     DataToLabView.floatData[6] = (float)statePos;
1212     DataToLabView.floatData[7] = LADARfront;
1213     LVsenddata[0] = '*'; // header for LVdata
1214     LVsenddata[1] = '$';
1215     for (i=0;i<LVNUM_TOFROM_FLOATS*4;i++) {
1216         if (i%2==0) {
1217             LVsenddata[i+2] = DataToLabView.rawData[i/2] & 0xFF;
1218         } else {
1219             LVsenddata[i+2] = (DataToLabView.rawData[i/2]>>8) & 0xFF;
1220         }
1221     }
1222     serial_sendSCID(&SerialID, LVsenddata, 4*LVNUM_TOFROM_FLOATS + 2);
1223 }
1224
1225 }
1226 timecount++;
1227 if((timecount%200) == 0)
1228 {
1229     if(doneCal == 0) {
1230         GpioDataRegs.GPATOGGLE.bit.GPIO31 = 1; // Blink Blue LED while calibrating
1231     }
1232     GpioDataRegs.GPBTOGGLE.bit.GPIO34 = 1; // Always Block Red LED
1233     UARTPrint = 1; // Tell While loop to print
1234 }
1235
1236 SpibRegs.SPIFFCT.bit.TXDLY = 16;
1237 CurrentChip = DAN28027;
1238 SpibRegs.SPIFRX.bit.RXFIL = 3;
1239 GpioDataRegs.GPACLEAR.bit.GPIO29 = 1;
1240 SpibRegs.SPITXBUF = 0x00DA;
1241 SpibRegs.SPITXBUF = EPwm1A_F28027;
1242 SpibRegs.SPITXBUF = EPwm2A_F28027;
1243
1244
1245 //#####
1246 //
1247 // Restore registers saved:
1248 //
1249 DINT;
1250 PieCtrlRegs.PIEIER12.all = TempPIEIER;
1251
1252 GpioDataRegs.GPBCLEAR.bit.GPIO61 = 1;

```

```

1253 }
1254
1255 //
1256 // Connected to PIEIER12_10 (use MINT12 and MG12_10 masks):
1257 //
1258 _interrupt void SWI2_MiddlePriority(void) // RAM_CORRECTABLE_ERROR
1259 {
1260     // Set interrupt priority:
1261     volatile Uint16 TempPIEIER = PieCtrlRegs.PIEIER12.all;
1262     IER |= M_INT12;
1263     IER &= MINT12; // Set "global" priority
1264     PieCtrlRegs.PIEIER12.all &= MG12_10; // Set "group" priority
1265     PieCtrlRegs.PIEACK.all = 0xFFFF; // Enable PIE interrupts
1266     __asm(" NOP");
1267     EINT;
1268
1269     //#####
1270     // Insert SWI ISR Code here.....
1271     // LED1
1272     GpioDataRegs.GPATOGGLE.bit.GPIO22 = 1;
1273     if (LADARpingpong == 1) {
1274         // LADARrightfront is the min of dist 52, 53, 54, 55, 56
1275         LADARrightfront = 19; // 19 is greater than max feet
1276         for (LADARi = 52; LADARi <= 56; LADARi++) {
1277             if (ladar_data[LADARi].distance_ping < LADARrightfront) {
1278                 LADARrightfront = ladar_data[LADARi].distance_ping;
1279             }
1280         }
1281         // LADARfront is the min of dist 111, 112, 113, 114, 115
1282         LADARfront = 19;
1283         for (LADARi = 111; LADARi <= 115; LADARi++) {
1284             if (ladar_data[LADARi].distance_ping < LADARfront) {
1285                 LADARfront = ladar_data[LADARi].distance_ping;
1286             }
1287         }
1288         LADARxoffset = ROBOTps.x + (LADARps.x*cosf(ROBOTps.theta)-LADARps.y*sinf(ROBOTps.theta));
1289         LADARyoffset = ROBOTps.y + (LADARps.x*sinf(ROBOTps.theta)+LADARps.y*cosf(ROBOTps.theta));
1290         for (LADARi = 0; LADARi < 228; LADARi++) {
1291
1292             ladar_pts[LADARi].x = LADARxoffset + ladar_data[LADARi].distance_ping*cosf(ladar_data[LADARi].angle + ROBOTps.theta);
1293             ladar_pts[LADARi].y = LADARyoffset + ladar_data[LADARi].distance_ping*sinf(ladar_data[LADARi].angle + ROBOTps.theta);
1294
1295         }
1296     } else if (LADARpingpong == 0) {
1297         // LADARrightfront is the min of dist 52, 53, 54, 55, 56
1298         LADARrightfront = 19; // 19 is greater than max feet
1299         for (LADARi = 52; LADARi <= 56; LADARi++) {
1300             if (ladar_data[LADARi].distance_pong < LADARrightfront) {
1301                 LADARrightfront = ladar_data[LADARi].distance_pong;
1302             }
1303         }
1304         // LADARfront is the min of dist 111, 112, 113, 114, 115
1305         LADARfront = 19;
1306         for (LADARi = 111; LADARi <= 115; LADARi++) {
1307             if (ladar_data[LADARi].distance_pong < LADARfront) {
1308                 LADARfront = ladar_data[LADARi].distance_pong;
1309             }
1310         }
1311         LADARxoffset = ROBOTps.x + (LADARps.x*cosf(ROBOTps.theta)-LADARps.y*sinf(ROBOTps.theta - PI/2.0));
1312         LADARyoffset = ROBOTps.y + (LADARps.x*sinf(ROBOTps.theta)-LADARps.y*cosf(ROBOTps.theta - PI/2.0));
1313         for (LADARi = 0; LADARi < 228; LADARi++) {
1314
1315             ladar_pts[LADARi].x = LADARxoffset + ladar_data[LADARi].distance_pong*cosf(ladar_data[LADARi].angle + ROBOTps.theta);
1316             ladar_pts[LADARi].y = LADARyoffset + ladar_data[LADARi].distance_pong*sinf(ladar_data[LADARi].angle + ROBOTps.theta);
1317
1318         }
1319     }
1320 }
1321
1322
1323
1324 //#####
1325 //
1326 // Restore registers saved:
1327 //
1328 DINT;
1329 PieCtrlRegs.PIEIER12.all = TempPIEIER;
1330
1331 }
1332
1333 //
1334 // Connected to PIEIER12_11 (use MINT12 and MG12_11 masks):
1335 //
1336 _interrupt void SWI3_LowestPriority(void) // FLASH_CORRECTABLE_ERROR
1337 {
1338     // Set interrupt priority:
1339     volatile Uint16 TempPIEIER = PieCtrlRegs.PIEIER12.all;
1340     IER |= M_INT12;
1341     IER &= MINT12; // Set "global" priority
1342     PieCtrlRegs.PIEIER12.all &= MG12_11; // Set "group" priority
1343     PieCtrlRegs.PIEACK.all = 0xFFFF; // Enable PIE interrupts
1344     __asm(" NOP");
1345     EINT;
1346
1347     //#####
1348     // Insert SWI ISR Code here.....
1349
1350     RCangle = readEncWheel();
1351     setEPWM3A_RCServo(RCangle); //RCangle is a value from -90 to 90
1352     setEPWM3B_RCServo(RCangle); //RCangle is a value from -90 to 90
1353     setEPWM5A_RCServo(RCangle); //RCangle is a value from -90 to 90
1354     setEPWM5B_RCServo(RCangle); //RCangle is a value from -90 to 90
1355     setEPWM6A_RCServo(RCangle); //RCangle is a value from -90 to 90
1356
1357     //#####
1358     //
1359     // Restore registers saved:
1360     //
1361     DINT;
1362     PieCtrlRegs.PIEIER12.all = TempPIEIER;
1363
1364 }
1365
1366 // ---- code for CAN start here ----

```

```

1367 interrupt void can_isr(void)
1368 {
1369     int i = 0;
1370
1371     uint32_t status;
1372
1373     //
1374     // Read the CAN interrupt status to find the cause of the interrupt
1375     //
1376     status = CANGetInterruptCause(CANB_BASE);
1377
1378     //
1379     // If the cause is a controller status interrupt, then get the status
1380     //
1381     if(status == CAN_INT_INT0ID_STATUS)
1382     {
1383         //
1384         // Read the controller status. This will return a field of status
1385         // error bits that can indicate various errors. Error processing
1386         // is not done in this example for simplicity. Refer to the
1387         // API documentation for details about the error status bits.
1388         // The act of reading this status will clear the interrupt.
1389         //
1390         status = CANgetStatus(CANB_BASE);
1391     }
1392
1393     //
1394     // Check if the cause is the receive message object 2
1395     //
1396     //
1397     else if(status == RX_MSG_OBJ_ID_1)
1398     {
1399         //
1400         // Get the received message
1401         //
1402         CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_1, rxMsgData);
1403
1404         for(i = 0; i<2; i++)
1405         {
1406             dis_raw_1[i] = rxMsgData[i];
1407         }
1408
1409         dis_1 = 256*dis_raw_1[1] + dis_raw_1[0];
1410
1411         measure_status_1 = rxMsgData[2];
1412
1413         //
1414         // Getting to this point means that the RX interrupt occurred on
1415         // message object 2, and the message RX is complete. Clear the
1416         // message object interrupt.
1417         //
1418         CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_1);
1419
1420         //
1421         // Increment a counter to keep track of how many messages have been
1422         // received. In a real application this could be used to set flags to
1423         // indicate when a message is received.
1424         //
1425         rxMsgCount_1++;
1426
1427         //
1428         // Since the message was received, clear any error flags.
1429         //
1430         errorFlag = 0;
1431     }
1432
1433     else if(status == RX_MSG_OBJ_ID_2)
1434     {
1435         //
1436         // Get the received message
1437         //
1438         CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_2, rxMsgData);
1439
1440         for(i = 0; i<2; i++)
1441         {
1442             dis_raw_2[i] = rxMsgData[i];
1443         }
1444
1445         dis_2 = 256*dis_raw_2[1] + dis_raw_2[0];
1446
1447         measure_status_2 = rxMsgData[2];
1448
1449         //
1450         // Getting to this point means that the RX interrupt occurred on
1451         // message object 2, and the message RX is complete. Clear the
1452         // message object interrupt.
1453         //
1454         CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_2);
1455
1456         //
1457         // Increment a counter to keep track of how many messages have been
1458         // received. In a real application this could be used to set flags to
1459         // indicate when a message is received.
1460         //
1461         rxMsgCount_2++;
1462
1463         //
1464         // Since the message was received, clear any error flags.
1465         //
1466         errorFlag = 0;
1467     }
1468
1469     else if(status == RX_MSG_OBJ_ID_3)
1470     {
1471         //
1472         // Get the received message
1473         //
1474         CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_3, rxMsgData);
1475
1476         for(i = 0; i<4; i++)
1477         {
1478             lightlevel_raw_1[i] = rxMsgData[i];
1479             quality_raw_1[i] = rxMsgData[i+4];
1480

```

```

1481     }
1482
1483     lightlevel_1 = ((256.0*256.0*256.0)*lightlevel_raw_1[3] + (256.0*256.0)*lightlevel_raw_1[2] + 256.0*lightlevel_raw_1[1] + lightlevel_raw_1[0])/65535;
1484     quality_1 = ((256.0*256.0*256.0)*quality_raw_1[3] + (256.0*256.0)*quality_raw_1[2] + 256.0*quality_raw_1[1] + quality_raw_1[0])/65535;
1485
1486
1487     //
1488     // Getting to this point means that the RX interrupt occurred on
1489     // message object 2, and the message RX is complete. Clear the
1490     // message object interrupt.
1491     //
1492     CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_3);
1493
1494     //
1495     // Since the message was received, clear any error flags.
1496     //
1497     errorFlag = 0;
1498 }
1499
1500
1501 else if(status == RX_MSG_OBJ_ID_4)
1502 {
1503     //
1504     // Get the received message
1505     //
1506     CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_4, rxMsgData);
1507
1508     for(i = 0; i<4; i++)
1509     {
1510         lightlevel_raw_2[i] = rxMsgData[i];
1511         quality_raw_2[i] = rxMsgData[i+4];
1512     }
1513
1514
1515     lightlevel_2 = ((256.0*256.0*256.0)*lightlevel_raw_2[3] + (256.0*256.0)*lightlevel_raw_2[2] + 256.0*lightlevel_raw_2[1] + lightlevel_raw_2[0])/65535;
1516     quality_2 = ((256.0*256.0*256.0)*quality_raw_2[3] + (256.0*256.0)*quality_raw_2[2] + 256.0*quality_raw_2[1] + quality_raw_2[0])/65535;
1517
1518     //
1519     // Getting to this point means that the RX interrupt occurred on
1520     // message object 2, and the message RX is complete. Clear the
1521     // message object interrupt.
1522     //
1523     CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_4);
1524
1525     //
1526     // Since the message was received, clear any error flags.
1527     //
1528     errorFlag = 0;
1529 }
1530
1531
1532
1533 //
1534 // If something unexpected caused the interrupt, this would handle it.
1535 //
1536 else
1537 {
1538     //
1539     // Spurious interrupt handling can go here.
1540     //
1541 }
1542
1543 //
1544 // Clear the global interrupt flag for the CAN interrupt line
1545 //
1546 CANclearGlobalInterruptStatus(CANB_BASE, CAN_GLOBAL_INT_CANINT0);
1547
1548 //
1549 // Acknowledge this interrupt located in group 9
1550 //
1551 InterruptclearACKGroup(INTERRUPT_ACK_GROUP9);
1552 }
1553 // ----- code for CAN end here -----

```